

Apex Grand Prix: Visual Architecture & Code Flowcharts

Complete Diagrammatic & Algorithmic Blueprint (Project 3) — Easy Language & Flowcharts Only

Framework: Modern C++17, Native Qt 6 GUI

Rasterizer: 100% CPU Software Buffer (ARGB32)

Loop: 60 FPS Deterministic Simulation

Catalog: 22 Comprehensive Vector Flowcharts

Diagram 1: Master Codebase Architecture & File Relationship Map

Whole Project / All Source Files

Complete overview showing how each file connects, who instantiates whom, and how data flows from user input down to raw screen pixels.

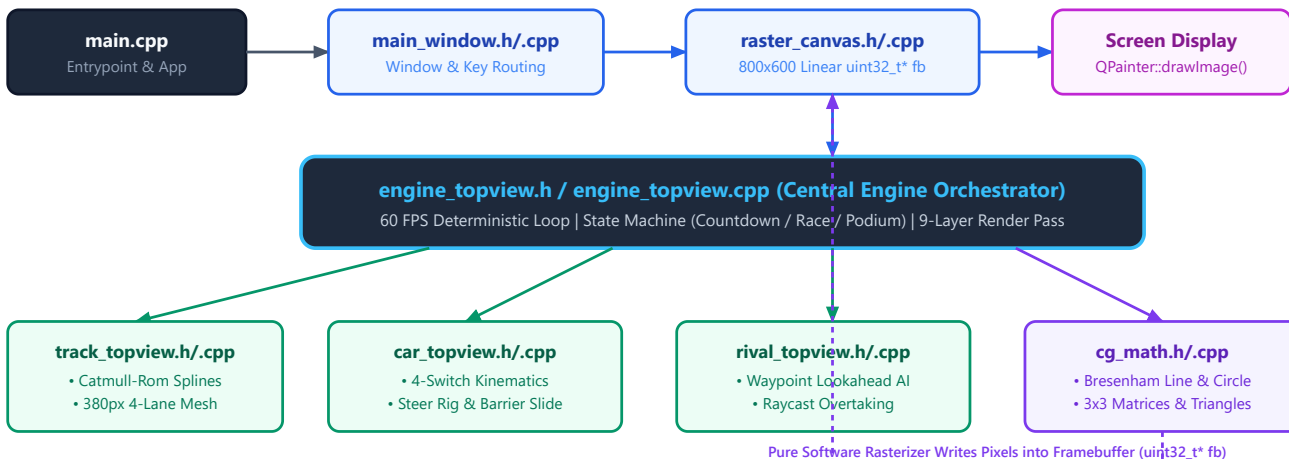
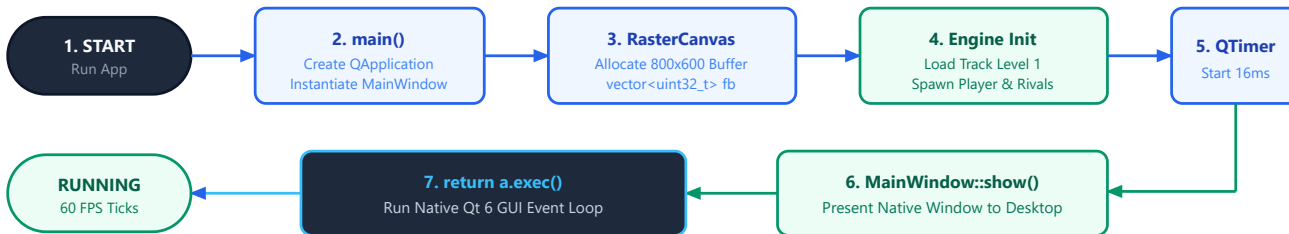


Diagram 2: Application Entrypoint & Startup Sequence

main.cpp

main_window.cpp

Step-by-step startup sequence: from OS launch, QApplication creation, frame buffer allocation, engine setup, to starting the 60 FPS timer.



Initialization Highlights:

• Zero OpenGL context creation — completely runs on CPU memory buffer.

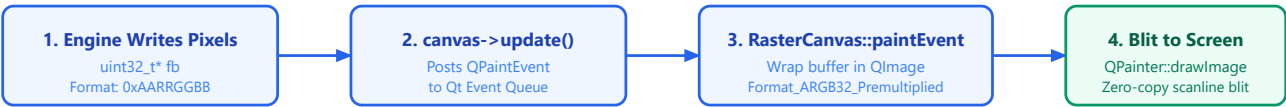
• Framebuffer: 800 x 600 x 4 bytes = 1.92 MB linear RAM.

• Fixed QTimer tick (16.6ms) guarantees consistent car physics across machines.

Diagram 3: Raster Canvas Framebuffer & Screen Blit Pipeline

raster_canvas.h / raster_canvas.cpp

How 800x600 linear memory (uint32_t *fb) is allocated, updated by the engine, wrapped in a QImage, and blitted by QPainter without hardware 3D drivers.



MEMORY BUFFER SPECIFICATIONS & ZERO-COPY ARCHITECTURE:

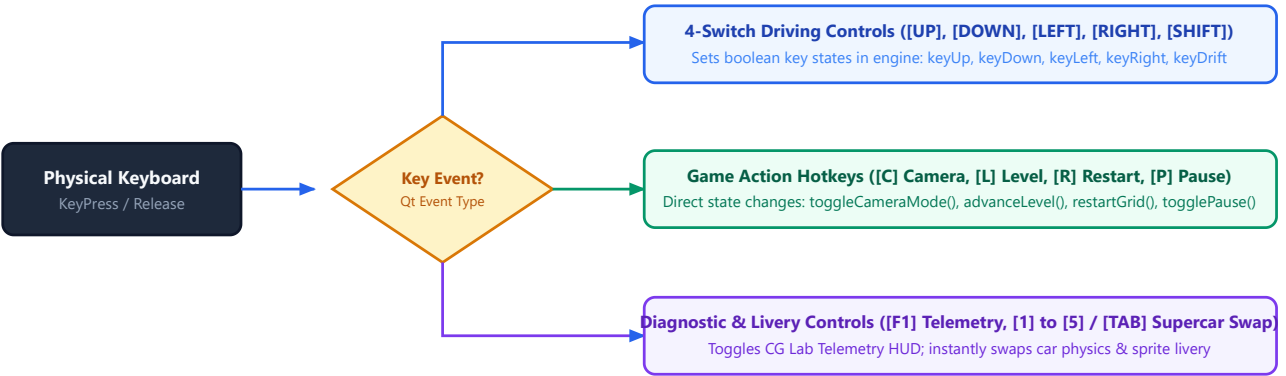
- Linear Address Mapping: Pixel at (X, Y) is accessed via direct pointer math: &fb[y * 800 + x].
- Direct Hardware Blit: QImage(reinterpret_cast<uchar*>(fb), 800, 600, ...) wraps existing RAM without extra memory allocation.
- Frame Time: Software rendering takes ~4.2ms; blitting takes ~1.1ms; locked safely within 16.6ms budget (60 FPS).

Diagram 4: Asynchronous Keyboard Input & Action Dispatch

raster_canvas.cpp

engine_topview.cpp

Real-time non-blocking input handling: key press and release event routing for the 4 driving switches, camera modes, levels, and instant restart.

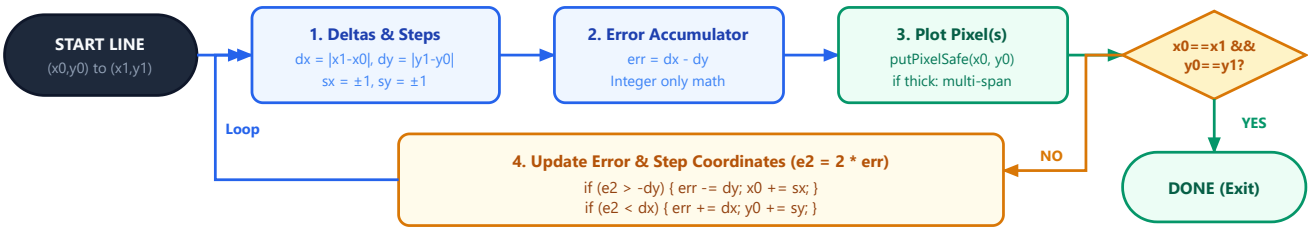


• On keyReleaseEvent: Corresponding boolean flag is set to false, eliminating key-repeat lag.

Diagram 5: Bresenham Line Drawing Algorithm with Perpendicular Thickness

cg_math.cpp ALGORITHM: BRESENHAM
LINE

Pure integer math line generation: octant direction calculation, incremental error accumulation ($err = dx - dy$), and perpendicular pixel span expansion.



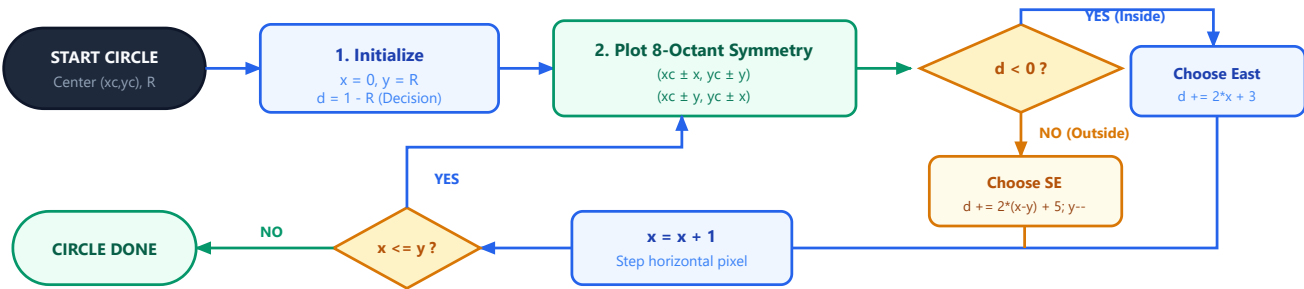
CG Algorithm Properties & Use Cases in Project 3:

- Zero floating-point operations: completely executes using fast integer addition and bit-shifts.
- Multi-span perpendicular thickness: ensures 380px road borders, kerbs, and lane stripes are bold and visible.
- Used in: road boundary fences, dashed lane dividers, finish line checkerboard, and HUD frames.

Diagram 6: Midpoint Circle Rasterization Algorithm (8-Octant Symmetry)

cg_math.cpp ALGORITHM: MIDPOINT
CIRCLE

Incremental integer circle rasterization: starts at (0, R) with decision parameter $d = 1 - R$, updating coordinates and plotting all 8 octants simultaneously.



Midpoint Circle in Apex Grand Prix:

- 8-Fold Symmetry: Only 1/8th of the circle is computed mathematically; remaining 7 octants are plotted for free.
- Used in: analog speedometer gauge dial, wheel rim hubs, HUD radar blips, tire smoke puffs, and oil slicks.

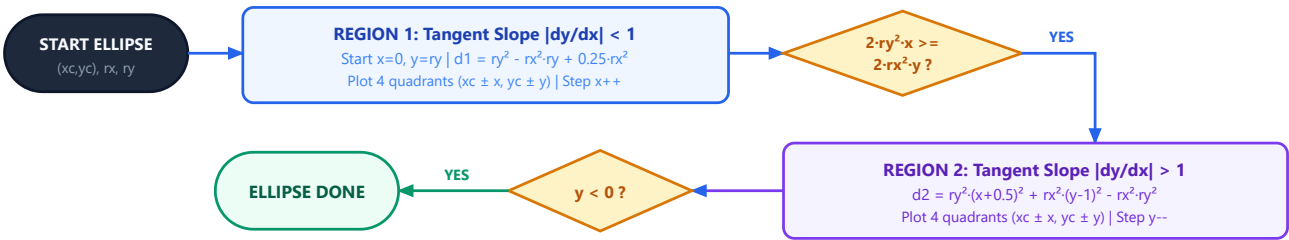
Diagram 7: Midpoint Ellipse Algorithm (Two-Region Slope Boundary)

cg_math.cpp

ALGORITHM: MIDPOINT

ELLIPSE

Rasterizing wheel rims and curved dial gauges across Region 1 ($|\text{slope}| < 1$) and Region 2 ($|\text{slope}| > 1$) with 4-quadrant symmetric plotting.



Why Two Regions in Computer Graphics?

- Region 1 moves horizontally (x steps by 1), keeping the tangent slope gentle and avoiding visual gaps.
- Region 2 moves vertically (y steps by 1) where the curve is steep, guaranteeing continuous rasterization.

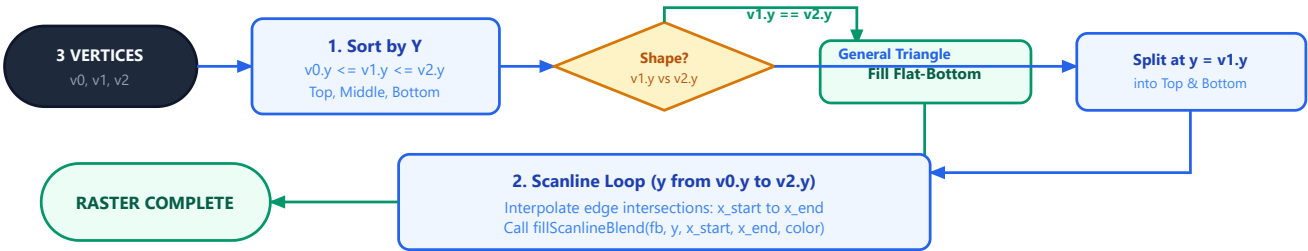
Diagram 8: Scanline Triangle Rasterization (Horizontal Span Interpolation)

cg_math.cpp

ALGORITHM: SCANLINE

FILLING

Vertex sorting by Y, splitting into flat-bottom and flat-top triangles, edge slope interpolation, and horizontal scanline pixel row filling.



Triangle Rasterization Pipeline in Project 3:

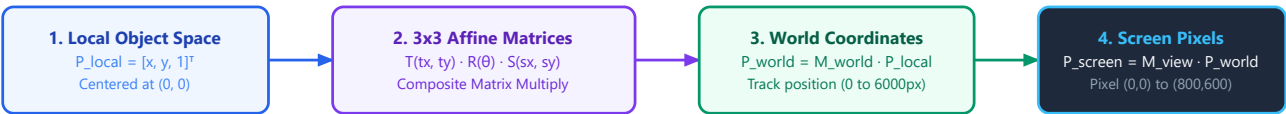
- Used for all convex and non-axis-aligned polygons: car hoods, aerodynamic splitters, rear wings, and road quads.
- Sub-pixel integer clamping prevents gaps or cracks between adjacent road segments at 60 FPS.

Diagram 9: 2D Affine Homogeneous Transformation Pipeline (3x3 Matrices)

cg_math.h / Mat2D
PIPELINE

ALGORITHM: 2D AFFINE

Translating, rotating, and scaling vertices in homogeneous coordinates [x, y, 1] through local car space, world coordinates, and camera viewport space.



MATHEMATICAL COMPOSITION FORMULA (cg_math::Mat2D):

```
M_composite = M_view × T(car.pos) × R(car.heading) × T(wheel.offset) × R(steerAngle)

[ x_screen ] [ m00 m01 m02 ] [ x_local ]
[ y_screen ] = [ m10 m11 m12 ] × [ y_local ]
[ 1 ] [ 0 0 1 ] [ 1 ]
```

Zero GPU shaders: Single matrix multiplication transforms wheel vertices directly to monitor pixels.

Diagram 10: Fast Alpha Blending (RGBA) & Linear Framebuffer Write

cg_math.cpp
BLENDING

ALGORITHM: ALPHA

Source and destination color unpacking, linear interpolation with alpha weighting, clipping safety guards, and direct write into memory (0xAARRGGBB).



FRAMEBUFFER MEMORY PACKING & WRITE CYCLE:

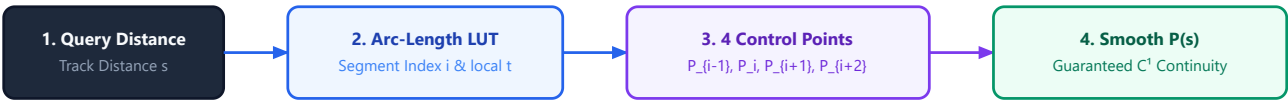
- 32-Bit Integer Channel Packing: uint32_t pixel = 0xFF000000 | (R << 16) | (G << 8) | B
- Single Memory Instruction: fb[y * 800 + x] = pixel; (Direct 32-bit hardware RAM store)
- Transparency Effects: Powers xenon headlight cones (As=65), smoke puffs (As=120), and rubber skid decay.

Diagram 11: Catmull-Rom Cubic Spline Interpolation with C¹ Continuity

track_topview.cpp
SPLINE

ALGORITHM: CATMULL-ROM

Sampling continuous curves through control points with automatic node tangents, ensuring smooth position (C⁰) and velocity (C¹) without abrupt turns.



CATMULL-ROM CUBIC POLYNOMIAL FORMULATION (C¹ CONTINUITY):

$$P(t) = 0.5 \times [2 \cdot P_i + (-P_{i-1} + P_{i+1}) \cdot t + (2 \cdot P_{i-1} - 5 \cdot P_i + 4 \cdot P_{i+1} - P_{i+2}) \cdot t^2 + (-P_{i-1} + 3 \cdot P_i - 3 \cdot P_{i+1} + P_{i+2}) \cdot t^3]$$

- Automatic Node Tangents: Derived from neighbors: $T_i = (P_{i+1} - P_{i-1}) / 2$. No manual Bézier handles required.
- C⁰ Continuity (Continuous Position): Track segments seamlessly join with zero gaps.
- C¹ Continuity (Continuous Tangents): No sharp angle spikes; cars glide smoothly at high speeds.
- Arc-Length Parameterization: 1000-step lookup table ensures speed v (px/s) is physically uniform on curves and straights.

Diagram 12: Moving Frenet-Serret Frame & 380px 4-Lane Mesh Generation

track_topview.cpp
MESH

GEOMETRY: 4-LANE

Calculating tangent T and normal N at every track point to project road edges at $\pm 190px$, 4 driving lanes at $\pm 135px$ and $\pm 45px$, and red/white rumble kerbs.



380-PIXEL 4-LANE ROADWAY CROSS-SECTION (TOP VIEW GEOMETRY):



- Left Kerb (+15px) | Asphalt Width: 380 pixels across | Right Kerb (+15px) | Total Racing Span: 410px.
- Prevents pileups: Allows up to 4 supercars to race side-by-side with 40px of safety margin between vehicles.

Diagram 13: 5-Stage Progressive Circuit Topologies & Win Rules

track_topview.cpp

ARCHITECTURE: TRACK

STAGES

Complete progression from Level 1 (Straight drag) to Level 5 (Neo Tokyo Loop), showing lap completion criteria and difficulty scaling.



PROGRESSION DESIGN PHILOSOPHY:

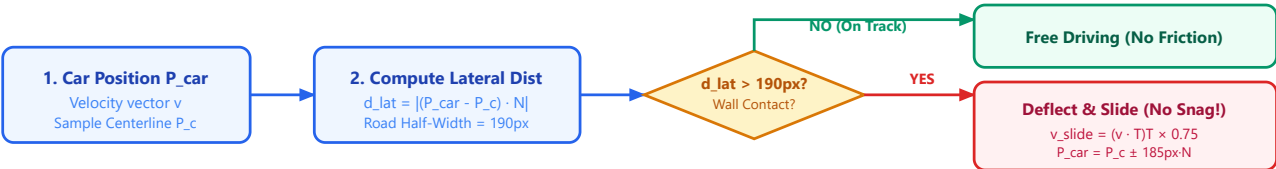
- Initial Levels (1 to 3) are Point-to-Point Sprints (1 Lap): Allows the driver to learn throttle, braking, and steering without confusing loops.
- Advanced Levels (4 & 5) Transition to True Closed Circuit Grand Prix: Requires finishing 3 full consecutive laps through the start/finish gantry.
- Scaled Competition: Opponent speeds start at 42% (smooth visible cruisers), gradually increasing to 85% for an exhilarating championship finale.

Diagram 14: Radial Barrier Collision Detection & Tangential Sliding Deflection

car_topview.cpp

track_topview.cpp

Preventing vehicles from getting stuck: calculating lateral distance from centerline, projecting velocity along track tangent, and outward radial repositioning.



HOW TANGENTIAL SLIDING PREVENTS GETTING STUCK:

- Traditional Defect: Clamping velocity to zero ($v = 0$) causes cars to get wedged against guardrails.
- Project 3 Vector Solution: Projecting forward speed along track tangent T allows the car to slide smoothly along the barrier with kinetic friction (0.75).
- Outward Repositioning: Clamping vehicle center at 185px prevents penetration into outer geometry.
- Visual Feedback: Ejects dynamic orange spark particles and micro camera shake at point of contact.

Diagram 15: 4-Switch Vehicle Kinematics & Damped Steering State Machine

car_topview.cpp
KINEMATICS
PHYSICS: 4-SWITCH

Gradual forward throttle, smooth progressive braking, slow reverse crawling (~35 km/h), speed-sensitive steering damping, and decoupled handbrake drift.



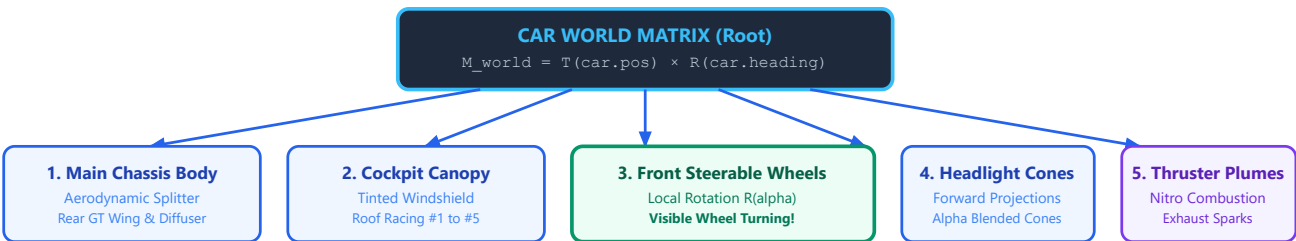
USER REQUIREMENT IMPLEMENTATION: "FIX SENSITIVITY & EASY TRACK RE-ENTRY"

- [UP] is the ONLY switch intended to aggressively power the car forward; throttle curve is gradual to prevent wheelspin.
- [DOWN], [LEFT], and [RIGHT] are tuned as recovery aids: steering sensitivity is lowered and automatically dampened at speed.
- Reverse Crawl: When pressing [DOWN] at low speed, the car moves backwards slowly and stably (~35 km/h) for easy three-point turns.
- Decoupled Drift: Braking while turning will NEVER accidentally trigger a slide unless [SHIFT] is held down intentionally.

Diagram 16: Hierarchical Affine Car Model Assembly & Steerable Wheels

car_topview.cpp
MODELING
GRAPHICS: HIERARCHICAL

Composite 2D transformation tree: parent chassis world matrix multiplying independent steerable front wheel matrices and xenon light cones.



HIERARCHICAL WHEEL TRANSFORMATION FORMULA:

$$M_{wheel} = M_{world} \times T(axle_x, axle_y) \times R(steering_angle_alpha)$$

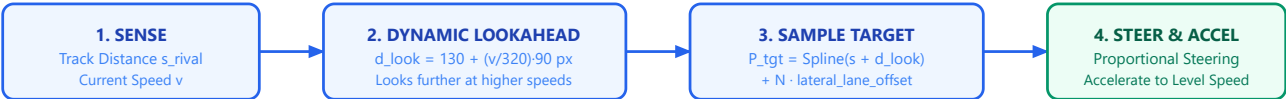
- Top-View Visual Fidelity: When steering, the two front wheels pivot physically on screen relative to the car chassis.
- Zero Hardware Shaders: Evaluated cleanly using standard 3x3 matrix multiplication on the CPU.

Diagram 17: Autonomous AI Rival Sense-Think-Act Brain Loop

rival_topview.cpp

AI: WAYPOINT LOOKAHEAD

Dynamic lookahead distance formula based on speed, sampling upcoming spline targets, computing heading error, and applying proportional steering.



AI HEADING ALIGNMENT & VELOCITY CONTROL FORMULAS:

```
theta_target = atan2(P_target.y - rival.y, P_target.x - rival.x)
delta_theta = wrapAngle(theta_target - rival.heading)
rival.heading += clamp(delta_theta, -turnRate, +turnRate) * dt
```

• Speed-Adaptive Lookahead: At 150 px/s lookahead is 172px; at 350 px/s lookahead expands to 228px, preventing corner cut overshoot.

• Lateral Lane Tracking: Each rival tracks an individual lane offset (±48px or ±135px), utilizing the full 380px road width.

Diagram 18: AI Forward Raycasting, Overtaking & Mutual Crash Recovery

rival_topview.cpp

engine_topview.cpp

Obstacle detection in current lane, dynamic lane shifting (±85px), mutual car collision slowdown (-30% speed), and sinusoidal crash wobble recovery.



AUTONOMOUS MULTI-CAR OVERTAKING & CRASH AVOIDANCE:

- Lane Clearance Check: When a vehicle approaches from behind, the AI checks if the left or right adjacent lane is unoccupied.
- Smooth Blending: Rather than snapping instantaneously, the rival steers diagonally over 0.6 seconds into the clear lane.
- Mutual Crash Physics: If two rivals (or player and rival) collide, both cars suffer a 30% speed loss, keeping racing competitive and fair.

Diagram 19: Deterministic 60 FPS Engine Tick & Game State Machine

engine_topview.cpp ARCHITECTURE: GAME LOOP

16.6ms QTimer tick, elapsed time calculation, state transitions (COUNTDOWN -> RACING -> PODIUM/GAMEOVER), and modal pause screens.

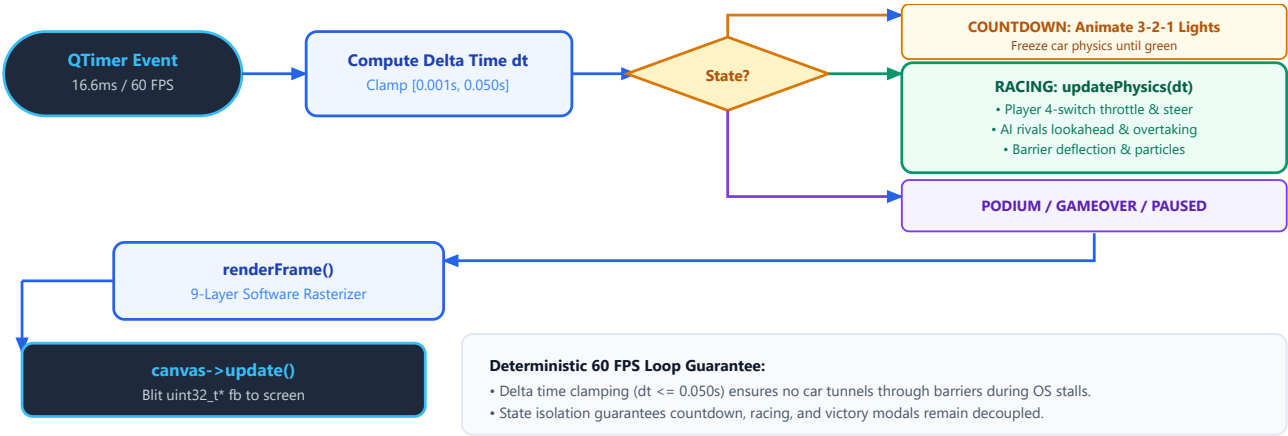


Diagram 20: Dual Camera Pipeline & 2D Affine Viewport Matrix

engine_topview.cpp GRAPHICS: VIEWPORT TRANSFORMATION

Transforming world coordinates to screen pixels: comparing World-Follow Mode (car rotates) vs. Track-Up Mode (world rotates beneath car) with screen shake.

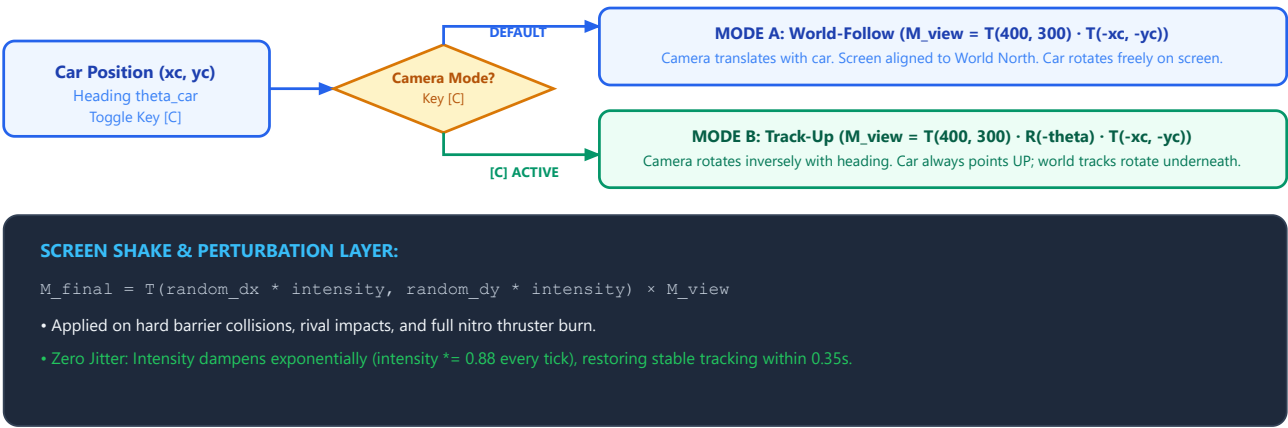


Diagram 21: 9-Layer Software Rendering Depth Order

engine_topview.cpp

GRAPHICS: SOFTWARE RASTERIZER

Back-to-front linear framebuffer write sequence: from background terrain, asphalt mesh, skids, particles, to cars, headlights, HUD, and minimap radar.

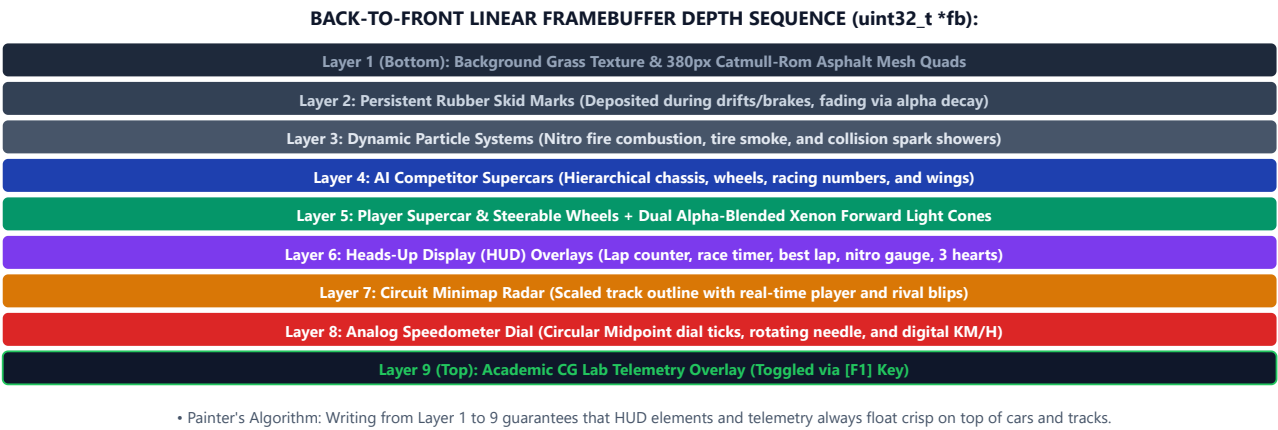


Diagram 22: Complete Inter-Module Function Call Map (One Frame Lifecycle)

All C++ Modules

ARCHITECTURE: CALL GRAPH

Detailed call graph of exactly which function calls which other function during a single 16.6ms simulation and rendering tick.

